

UNIVERSIDAD DE LA HABANA
FACULTAD DE MATEMÁTICA Y COMPUTACIÓN

Un compilador para

HULK

*Diseño, implementación y extensión de un
lenguaje de programación orientado a objetos*

REPORTE DE PROYECTO FINAL
Asignaturas de Compilación y Lenguajes de Programación

CURSO	2025–2026
CARRERA	Ciencias de la Computación
AÑO	Tercer año
BACKEND	LLVM 17 (código nativo x86-64)
STACK	RUST 1.77+
EXTENSIÓN	Vectores y expresiones generadoras

Resumen

SE presenta un compilador *ahead of time* para HULK (*Havana University Language for Compilers*), un lenguaje multiparadigma estáticamente tipado, orientado a expresiones y con un sistema de objetos basado en herencia simple y protocolos estructurales. El compilador está escrito íntegramente en RUST (edición 2024) y produce ejecutables nativos x86-64 mediante LLVM 17, accedido a través de la biblioteca *inkwell*. El *frontend* (analizadores léxico, sintáctico y semántico) y el *backend* (generación de LLVM IR, optimización y enlace con un *runtime* en C) se implementan sin recurrir a generadores externos de *lexers* ni *parsers*: el analizador léxico se construye sobre autómatas finitos no deterministas obtenidos por la construcción clásica de Thompson, y el analizador sintáctico es un motor LALR(1) cuyas tablas se generan en tiempo de inicialización a partir de una definición declarativa de la gramática de HULK.

La implementación cubre todas las características nucleares del lenguaje —expresiones aritméticas y de cadena, control de flujo expresivo, funciones de primer orden, tipos con herencia simple, protocolos con conformidad estructural y los operadores de prueba y conversión de tipos *is/as*—, e incorpora como aporte propio un **sistema de vectores con expresiones generadoras**. Esta extensión añade un constructor de tipos paramétrico `Vector<T>` al sistema de tipos, tres nuevas formas sintácticas (literales, generadores e indexación) y una semántica operacional ansiosa con verificación de límites en tiempo de ejecución. Su diseño está inscrito en la familia clásica de las *list comprehensions* de Haskell [7] y Python, las *for-comprehensions* de Scala [8] y los *ranges and views* de C++20, y se compara con cada una de ellas para justificar las decisiones de diseño tomadas.

El cuerpo del documento analiza con detalle las cinco fases del compilador, expone las decisiones técnicas relevantes junto a sus alternativas descartadas, formaliza la sintaxis y la semántica del lenguaje implementado mediante reglas de inferencia tipográfica y semántica operacional de gran paso, y reporta los resultados de validación obtenidos al ejecutar la suite de pruebas sobre el binario producido.

Índice general

Resumen	1
1 Introducción	5
1.1 El lenguaje HULK	5
1.2 Principios de diseño del compilador	6
2 Arquitectura del compilador	8
2.1 El pipeline de compilación	8

2.2	Las cuatro fases del <i>frontend</i>	9
2.3	Las fases del <i>backend</i>	10
2.4	Elección del lenguaje anfitrión: RUST	11
3	Análisis léxico	12
3.1	Arquitectura: lexer basado en NFA	12
3.1.1	Construcción de Thompson	12
3.1.2	Ejecución sin determinización	12
3.1.3	El MasterNFA y la regla del máximo prefijo	13
3.2	Conjunto de tokens reconocidos	13
3.3	Secuencias de escape: limitación conocida	13
3.4	Recuperación de errores	14
4	Análisis sintáctico	15
4.1	Parser LALR(1) implementado desde cero	15
4.2	Pipeline del parser	15
4.2.1	Definición de la gramática	15
4.2.2	Calculo de FIRST y FOLLOW	16
4.2.3	Construcción de la tabla LALR	16
4.2.4	Motor de parseo	17
4.3	Jerarquía de precedencias	17
4.4	El árbol sintáctico abstracto	17
5	Análisis semántico	19
5.1	Estrategia en dos pasadas	19
5.2	Sistema de tipos	20
5.2.1	Categorías de tipo	20
5.2.2	Relación de conformidad	20
5.2.3	Unificación de tipos en condicionales: el ancestro común más cercano	21
5.3	Funciones y constantes predefinidas	21
5.4	Reporte de errores semánticos	22
6	Generación de código	23
6.1	LLVM IR mediante <i>inkwell</i>	23
6.2	Representación de valores: el enum CgValue	23
6.3	Representación de objetos en memoria	24
6.4	Tablas de métodos virtuales (vtables)	24
6.4.1	Despacho de métodos a través de un protocolo	25
6.5	Etiquetas de tipo en orden DFS	26
6.6	El operador as: downcast con verificación	26
6.7	La palabra clave base()	27
6.8	Pipeline de optimización	27
7	Runtime y compilación final	28
7.1	La biblioteca de runtime en C	28
7.2	Layout en memoria de vectores y rangos	28
7.3	Pipeline AOT completo	29
8	Extensión del lenguaje: vectores con expresiones generadoras	30

8.1	Motivación	30
8.2	Definición formal de la sintaxis	31
8.2.1	Vectores explícitos	31
8.2.2	Expresiones generadoras	31
8.2.3	Indexación	32
8.3	Definición formal de la semántica	32
8.3.1	Reglas de tipado	32
8.3.2	Semántica operacional de gran paso	33
8.4	Realización en el código	33
8.4.1	Modificaciones al lexer	33
8.4.2	Modificaciones al parser	33
8.4.3	Modificaciones al semántico	34
8.4.4	Modificaciones al codegen	34
8.5	Análisis comparativo con otros lenguajes	35
8.5.1	Notación matemática de conjuntos	35
8.5.2	Comparación con Python	35
8.5.3	Comparación con Haskell	36
8.5.4	Comparación con Scala	36
8.5.5	Comparación con C++20 ranges	36
8.5.6	Resumen comparativo	37
8.6	Decisiones de diseño de la extensión	37
8.6.1	Por que evaluación ansiosa y no perezosa	37
8.6.2	Por que la barra simple como separador	38
8.6.3	Por que vectores inmutables (tamaño fijo)	38
9	Cobertura de la especificación	39
9.1	Tabla de cobertura por sección	39
9.2	Ejemplos representativos	40
9.2.1	Herencia y polimorfismo	40
9.2.2	Protocolos con conformidad estructural	40
9.2.3	Vectores y generadores	40
10	Decisiones de diseño significativas	41
10.1	LALR(1) artesanal frente a generador externo	41
10.2	LLVM como backend vs interprete vs VM propia	41
10.3	Vtables vs tagged union vs boxed types	42
10.4	Type tags DFS pre-orden vs class objects	42
10.5	Enlace dinámico vs estático de LLVM	42
11	Evaluación y pruebas	43
11.1	Estructura de la suite de pruebas	43
11.2	Programas de prueba completos	43
11.3	Contrato de errores	43
12	Limitaciones conocidas y trabajo futuro	45
12.1	Limitaciones actuales	45
12.2	Trabajo futuro	46

13 Conclusiones	47
------------------------	-----------

Bibliografía	49
---------------------	-----------

1

Introducción

LA traducción de programas escritos en un lenguaje de alto nivel a código nativo ejecutable es uno de los procesos fundamentales de la informática contemporánea y la actividad sobre la que se sostienen prácticamente todas las herramientas modernas de desarrollo de *software*. Tras cinco décadas de evolución la disciplina ha consolidado un *pipeline* de fases bien diferenciadas —análisis léxico, sintáctico y semántico, generación de código intermedio, optimización y emisión de código máquina— cuyos fundamentos teóricos están extensamente documentados en la literatura clásica [1, 14, 13], pero cuya implementación concreta para un lenguaje real exige resolver una gran cantidad de problemas prácticos no triviales: representación de tipos algebraicos en memoria, despacho de métodos en presencia de polimorfismo, propagación de tipos inferidos a través de fases independientes, integración con *backends* maduros como LLVM [4], manejo de errores con localización precisa, y un largo etcétera.

Este reporte describe **el diseño y la construcción de un compilador completo** para HULK (*Havana University Language for Kompilers*), un lenguaje multiparadigma de propósito didáctico cuya especificación combina elementos del paradigma funcional, imperativo y orientado a objetos. El compilador toma como entrada un fichero fuente `.hulk` y produce un ejecutable nativo Linux x86-64 que el sistema operativo puede cargar y ejecutar sin más dependencias externas que `libc` y `libm`. Toda la implementación está escrita en RUST salvo una pequeña biblioteca de soporte en tiempo de ejecución escrita en C, y utiliza LLVM 17 como infraestructura para la optimización y la emisión de código máquina.

1.1 El lenguaje HULK

HULK es un lenguaje estáticamente tipado, orientado a expresiones, con un sistema de objetos basado en herencia simple y protocolos estructurales. Tres rasgos lo definen como objetivo de compilación particularmente interesante.

Expresividad funcional. En HULK no existe la noción de *sentencia*: todas las construcciones del lenguaje son expresiones que producen un valor. Un `if-elif-else` no ejecuta ramas distintas con efecto de lado; *retorna* el valor de la rama elegida. Un bucle `while` retorna el valor de su última iteración. Un bloque $\{ e_1; e_2; \dots; e_n \}$ retorna e_n . Esta uniformidad obliga al verificador de tipos a calcular el tipo de cada construcción compuesta como el *ancestro común más cercano* de los tipos de sus subexpresiones, y obliga al generador de código a sintetizar nodos ϕ en cada punto de unión del grafo de flujo.

Tipado nominal con protocolos estructurales. La jerarquía de tipos definidos por el usuario sigue el modelo nominal clásico (dos tipos son iguales si tienen el mismo nombre), con herencia simple y sobrescritura de métodos. Sobre esa jerarquía se superpone un segundo mecanismo de polimorfismo basado en *protocolos*: tipos abstractos que declaran firmas de métodos sin cuerpo y que un tipo concreto *satisface implícitamente* con sólo declarar métodos compatibles —sin necesidad de mención explícita de la relación—. La conformidad estructural se rige por las reglas estándar de varianza (covarianza en retornos, contravarianza en parámetros) [6], lo que la convierte en una construcción rica desde el punto de vista del sistema de tipos.

Inferencia local de tipos. Las anotaciones de tipo son opcionales en parámetros, atributos e inicializadores de let. Cuando se omiten, el verificador de tipos debe inferir el tipo correcto mediante un análisis bidireccional: las firmas conocidas restringen los tipos hacia abajo, los usos restringen los tipos hacia arriba. La inferencia debe operar también en presencia de *recursión mutua* entre funciones y entre tipos, lo que obliga a separar la recolección de declaraciones de la verificación de los cuerpos.

1.2 Principios de diseño del compilador

El compilador descrito aquí está construido en torno a cinco principios de diseño explícitos, todos ellos consecuencia de un mismo compromiso de fondo: priorizar la transparencia de la maquinaria interna sobre la conveniencia inmediata. Las herramientas modernas de generación de *frontends* (LALRPOP, ANTLR, Bison) y de *backends* (Cranefield, GCC plugins) permiten obtener un compilador funcional en muy pocas líneas, pero a costa de ocultar precisamente los algoritmos que el documento se propone analizar.

D1. Transparencia algorítmica.

Cada fase implementa de forma explícita los algoritmos canónicos asociados a ella: construcción de Thompson en el *lexer*, cálculo de FIRST y FOLLOW por punto fijo y construcción canónica de LR(1) con fusión LALR(1) en el *parser*, propagación bidireccional de tipos con dos pasadas en el analizador semántico, *lowering* estructurado a SSA en el generador de código. Ninguna fase delega su núcleo algorítmico en una biblioteca externa.

D2. Compilación a código nativo, no interpretación.

El compilador es un traductor estricto: produce un binario ELF x86-64 enlazable, no un interpretador del AST ni una máquina virtual ad-hoc. La intención es validar la implementación contra la métrica más exigente posible —tiempos de ejecución comparables con el código C equivalente—, no contra una semántica abstracta.

D3. Seguridad de memoria del compilador mismo.

El compilador maneja estructuras de datos complejas con múltiples referencias cruzadas (tablas de símbolos jerárquicas, grafos de tipos, tablas LALR con cientos de estados). Se utilizó RUST como lenguaje anfitrión para eliminar de raíz una clase entera de errores que históricamente han plagado los compiladores escritos en C++: *dangling pointers*, *double frees*, lecturas de memoria inválida. El *wrapper inkwell* [10] aporta la misma garantía sobre el uso de la API C de LLVM.

D4. Reproducibilidad y *debuggability*.

Toda estructura de datos significativa —la gramática, la tabla LALR, el autómata, la jerarquía de tipos, el LLVM IR generado— es inspeccionable en tiempo de ejecución mediante volcados textuales legibles. La gramática se describe como una estructura de datos en memoria en lugar de un *DSL* externo, lo que permite imprimir cada producción con su identificador y depurar conflictos sin recompilar.

D5. Extensibilidad sintáctica y semántica.

El sistema de tipos, el AST y el generador de código fueron diseñados con anticipación a la introducción de extensiones. La extensión central documentada en este reporte —vectores con expresiones generadoras— añade tres formas sintácticas y un nuevo constructor de tipos paramétrico, y demuestra empíricamente que la organización modular es capaz de absorber esa adición sin reescrituras significativas.

2

Arquitectura del compilador

EL compilador adopta la organización en fases secuenciales que es habitual en la disciplina desde los primeros compiladores prácticos [18, 1]: el programa fuente atraviesa una sucesión de módulos, cada uno de los cuales lo transforma en una representación interna distinta hasta llegar al binario ejecutable. Este capítulo describe la arquitectura concreta del compilador implementado; los capítulos sucesivos entran en el detalle de cada fase.

2.1 El pipeline de compilación

La compilación es siempre una traducción *ahead of time*: el binario `hulk` recibe como argumento la ruta de un fichero `.hulk`, lo procesa por completo, escribe un fichero objeto en un directorio temporal y lo enlaza con la biblioteca de *runtime* mediante `gcc`, dejando en el directorio actual un ejecutable llamado `out.put`. El proceso es estrictamente secuencial: cada fase se ejecuta una sola vez, en orden, y sus errores se reportan según el código de salida fijado por el contrato de interfaz (1 para errores léxicos, 2 para sintácticos, 3 para semánticos).

La Figura 2.1 representa el pipeline tal como está implementado en `src/main.rs`. Las cajas verticales en línea continua corresponden a las fases que el compilador ejecuta en su orden de invocación; las cajas laterales en línea punteada representan las estructuras de datos que cada fase produce y la fase siguiente consume.

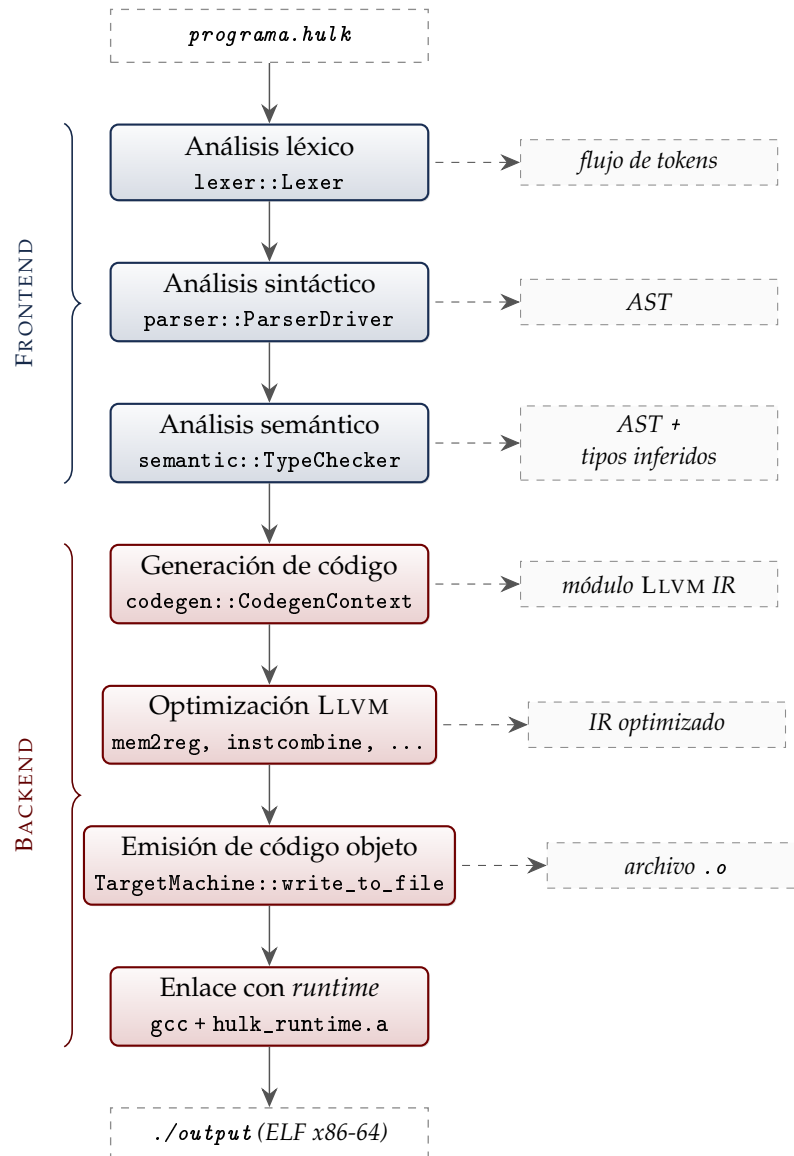


FIGURA 2.1. Pipeline de compilación implementado en `src/main.rs`. La columna central contiene las fases ejecutadas en orden; cada caja indica el módulo de Rust que realiza la transformación. Los recuadros laterales en línea punteada son las estructuras de datos que cada fase produce y la fase siguiente consume. Las llaves a la izquierda agrupan visualmente el *frontend* (azul) y el *backend* (rojo); la frontera entre ambos coincide con la producción del módulo de LLVM IR.

2.2 Las cuatro fases del frontend

El *frontend* comprende las tres primeras fases del compilador y se encarga de transformar el texto fuente en un AST anotado con tipos. La implementación reparte esta responsabilidad entre tres módulos de Rust con fronteras estrictas: `lexer`, `parser` y `semantic`. La Figura 2.1 muestra cada uno alimentando al siguiente sin retroalimentación.

Lexer. El módulo `lexer` reconoce tokens a partir del flujo de caracteres del fichero fuente. Su núcleo es un autómata finito no determinista construido por el algoritmo de Thompson a partir de las expresiones regulares declaradas para cada categoría léxica. Los autómatas de todos los patrones se combinan en un *MasterNFA*, del que el `lexer` toma decisiones por la

regla del máximo prefijo con desempate por prioridad. El resultado es un vector de tokens entregado por completo al *parser*; los tokens que encubren errores léxicos se marcan como `ERROR` sin interrumpir el reconocimiento, lo que permite reportar varios errores léxicos en una misma compilación.

Parser. El módulo `parser` consume la secuencia de tokens y construye el árbol sintáctico abstracto. Internamente se subdivide en cuatro submódulos: `grammar` define la gramática como una colección de producciones declarativas, `lalr` construye las tablas LALR(1) de `ACTION` y `GOTO` en tiempo de inicialización a partir de esa gramática, `engine` ejecuta el algoritmo *shift-reduce* sobre las tablas precalculadas, y `ast` define la jerarquía de nodos del árbol sintáctico abstracto. La construcción de las tablas —algoritmo canónico de DeRemer [3]— ocurre una sola vez al iniciarse el compilador.

Semantic. El módulo `semantic` verifica el AST y lo enriquece con información de tipos. Se compone de un submódulo de *sistema de tipos* —definiciones del enum `HulkType` y de la estructura `TypeHierarchy`—, una tabla de símbolos jerárquica con *scoping* estructurado, y un *type checker* que recorre el AST en dos pasadas: la primera registra las declaraciones de tipos, protocolos y funciones; la segunda visita los cuerpos, infiere los tipos ausentes y escribe los resultados en un mapa indexado por el identificador único de cada expresión. Al concluir, el análisis retorna una estructura con tres componentes: la jerarquía de tipos resuelta, las firmas de las funciones, y los tipos inferidos por expresión.

2.3 Las fases del backend

El *backend* —módulo `codegen`— traduce el AST tipado a un módulo de LLVM IR. La generación se orquesta en `codegen::jit`, que contiene los dos puntos de entrada del módulo: `execute_program_jit` para ejecución *just-in-time* usada en los tests, y `compile_to_binary` para la compilación AOT que produce el binario final. Ambas comparten el mismo flujo interno.

Lowering. La estructura `CodegenContext` encapsula el estado de la generación: el contexto, el módulo y el constructor de LLVM, una pila de tablas de símbolos con *scoping* léxico, el registro de *layouts* de structs y *vtables* por tipo, y los mapas de tipos heredados del analizador semántico. La operación `visit_program` construye el módulo LLVM en tres pasos sucesivos: declaración de las funciones externas del *runtime*, predeclaración de todas las funciones del programa (lo que habilita la recursión mutua sin reordenar el AST), y construcción de los *layouts* de tipos con asignación de *type tags* en orden DFS pre-orden. Sobre ese cimiento se compila el cuerpo de cada declaración (`lower_decl`) y, finalmente, la expresión de entrada del programa (`lower_expr`), envuelta en una función `__hulk_entry()` accesible desde la convención de llamada de C.

Optimización. El módulo LLVM producido se somete a un *pipeline* fijo de transformaciones aplicadas con el *new pass manager* de LLVM 17, declarado en `codegen::jit` como la cadena `"mem2reg, instcombine, reassociate, simplifycfg"`. `mem2reg` es el habilitador fundamental: promueve los patrones `alloca/store/load` emitidos por el *lowering* a registros virtuales SSA, lo que abre la puerta a las simplificaciones algebraicas y de control de

flujo aplicadas por los *passes* subsiguientes.

Emisión de código objeto. Tras la optimización, el módulo se escribe en un fichero objeto temporal mediante `TargetMachine::write_to_file`, configurado para la *triple nativa* de la máquina sobre la que se ejecuta el compilador (en la práctica, `x86_64-unknown-linux-gnu`).

Enlace. El compilador invoca finalmente `gcc` con tres argumentos: el fichero objeto recién emitido, el archivo `hulk_runtime.a` con las primitivas de *runtime*, y la bandera `-lm` para enlazar `libm`. El resultado es un binario ELF x86-64 ubicado en `./output` que el sistema operativo carga y ejecuta sin más dependencias dinámicas que `libc` y `libm`.

2.4 Elección del lenguaje anfitrión: RUST

Tres lenguajes constituyen alternativas razonables para implementar un compilador moderno con *backend* basado en LLVM: C, C++ y RUST. La elección recayó sobre RUST por cuatro razones técnicas concretas.

Seguridad de memoria sin coste de *garbage collection*. El compilador maneja estructuras de datos complejas con múltiples referencias cruzadas: tablas de símbolos jerárquicas, grafo de tipos con herencia, tabla de parseo LALR con cientos de estados. RUST garantiza la ausencia de *dangling pointers*, *double frees* y carreras de datos en tiempo de compilación, lo que elimina una clase entera de *bugs* históricamente difíciles de depurar en compiladores escritos en C++.

Sistema de tipos algebraico. Los enumerados con datos asociados (`enum`) y la verificación exhaustiva de patrones convierten la representación del AST en una tarea natural y segura. La ausencia de un caso en un `match` se reporta como error en tiempo de compilación, lo que evita bugs por casos faltantes.

Interfaz segura con LLVM. La biblioteca *inkwell* [10] encapsula la API C de LLVM en abstracciones de RUST que aprovechan el sistema de tipos para impedir operaciones inválidas (p. ej. construir una instrucción en un bloque básico ya terminado). Esto contrasta con el uso directo de `libLLVM` desde C++, donde tales errores producen *segfaults* difíciles de diagnosticar.

Ecosistema y herramientas. El gestor de paquetes `cargo`, la integración nativa con sistemas de pruebas y la disponibilidad de bibliotecas de calidad de producción permiten enfocar el esfuerzo en la lógica del compilador en lugar de en infraestructura.

3

Análisis léxico

EL primer paso de cualquier compilador consiste en transformar el flujo de caracteres del fichero fuente en una secuencia de *tokens* con significado. Esta fase se denomina *análisis léxico* o *tokenización*, y en este compilador se ha implementado mediante autómatas finitos no deterministas.

3.1 Arquitectura: lexer basado en NFA

3.1.1 Construcción de Thompson

La construcción clásica de Thompson [2] permite traducir una expresión regular cualquiera en un autómata finito no determinista con dos propiedades clave:

- exactamente un estado inicial y exactamente un estado de aceptación;
- las transiciones epsilon (ϵ) habilitan la composición modular de subautómatas sin alterar el alfabeto reconocido.

La construcción procede inductivamente sobre la estructura de la expresión regular, generando fragmentos NFA que se combinan según los operadores (concatenación, alternación, clausura). El tamaño del NFA resultante es lineal en el tamaño de la expresión regular original, lo que la hace adecuada para uso práctico.

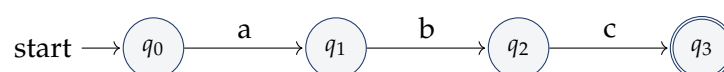


FIGURA 3.1. Fragmento NFA generado por Thompson para la expresión regular *abc*.

3.1.2 Ejecución sin determinización

El enfoque clásico convierte el NFA en un DFA antes de usarlo mediante la *construcción de subconjuntos*, lo que produce un autómata mas grande pero mas rápido de simular. En este compilador se ha optado por **ejecutar el NFA directamente**, calculando los subconjuntos de estados activos en cada paso. Esta decisión se justifica por:

- la simplicidad del código resultante;
- el tamaño modesto de los ficheros fuente HULK (decenas a miles de líneas);
- la ausencia de la fase de determinización, que podria duplicar exponencialmente el número de estados en el peor caso.

3.1.3 El MasterNFA y la regla del máximo prefijo

Para que el *lexer* reconozca todos los tipos de tokens a la vez, los NFA individuales de cada patron se combinan en un **MasterNFA**: un autómata con un nuevo estado inicial y transiciones epsilon hacia los estados iniciales de cada NFA componente. La Figura 3.2 ilustra el esquema.

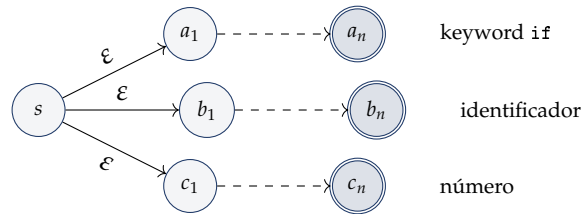


FIGURA 3.2. MasterNFA: estado inicial común con transiciones epsilon a los NFA individuales por patron.

Cuando varios estados de aceptación son alcanzables simultaneamente, la regla del *máximo prefijo* (*maximal munch*) selecciona el patron que reconoce la cadena mas larga. En caso de empate (por ejemplo, *if* podria interpretarse como *keyword* o como identificador), la **prioridad** numérica asignada a cada patron resuelve la ambigüedad: los *keywords* tienen mayor prioridad que los identificadores.

3.2 Conjunto de tokens reconocidos

HULK define 43 tipos de token agrupados en cinco categorías. La Cuadro 3.1 resume la clasificación.

TABLA 3.1. Conjunto de tokens del lenguaje HULK.

CATEGORIA	TOKENS
Palabras reservadas	let, in, if, elif, else, while, for, function, type, new, self, inherits, base, protocol, extends, is, as, true, false, null
Operadores aritméticos	+, -, *, /, %, ^, **
Operadores lógicos	&, , !
Operadores de comparación	==, !=, <, >, <=, >=
Operadores especiales	@, @@, :=, +=, -=, *=, /=, %=, =>, ->, ++, -
Delimitadores	(,), {, }, [,], ,, ;, :, .
Literales	números en notación decimal y flotante, cadenas entre comillas dobles, caracteres entre comillas simples, identificadores alfanumericos

3.3 Secuencias de escape: limitación conocida

Los literales de cadena se reconocen como una secuencia entre comillas dobles que admite cualquier carácter, salvo la propia comilla doble sin escapar. El *lexer* actual emite el lexema

en bruto, comillas incluidas, y la fase semántica retira las comillas envolventes para construir el nodo `Literal::String`. En esta versión del compilador, las secuencias de escape `\n`, `\t`, `\"` y `\\` no son descodificadas: cualquier contrabarra que aparezca en el cuerpo del literal se transmite tal cual al generador de código y, por tanto, al *runtime*. La consecuencia práctica es que un `print("\nhola")` imprime literalmente `\nhola` en lugar de un salto de línea seguido de `hola`. Esta omisión está documentada como limitación conocida en el Capítulo 12; su implementación es directa pero queda fuera del calendario del proyecto.

3.4 Recuperación de errores

Cuando el *lexer* encuentra un carácter que no inicia ningún patrón válido, emite un token especial `ERROR` en lugar de detener la compilación. Esto permite que el *parser* continúe avanzando y eventualmente reporte múltiples errores en una misma invocación. El formato del mensaje sigue el contrato oficial de la interfaz de evaluación:

```
! (línea,columna) LEXICAL: unexpected character 'X'
```

LISTADO 3.1. Formato estandar de error léxico.

4

Análisis sintáctico

UNA vez convertido el fuente en un flujo de tokens, la fase de análisis sintáctico verifica que dicho flujo se ajuste a la gramática del lenguaje y construye una representación estructurada del programa: el *árbol sintáctico abstracto* (AST).

4.1 Parser LALR(1) implementado desde cero

A pesar de la disponibilidad de generadores maduros como ANTLR, YACC, Bison o LALR-POP en el ecosistema RUST, el parser de este compilador está implementado manualmente sobre la base del principio de *transparencia algorítmica* (D1) enunciado en la introducción. Tres ventajas adicionales se derivan de esa decisión:

- **Inspeccionabilidad:** la gramática se representa como una estructura de datos RUST en lugar de procesarse desde un DSL textual externo. Esto permite imprimirla, recorrerla y analizar conflictos *shift/reduce* en tiempo de ejecución, sin pasos adicionales de compilación.
- **Cero dependencias externas:** el *build* del compilador no requiere invocar ni instalar generadores como YACC o Bison. La cadena de *build* se reduce a cargo *build*.
- **Control fino sobre la resolución de conflictos:** cuando la gramática introduce un conflicto deliberado —como el del separador `|` en las expresiones generadoras descritas en el Capítulo 8—, la resolución se programa directamente en el constructor de la tabla, sin necesidad de declarar precedencias en un metalenguaje externo.

4.2 Pipeline del parser

El módulo de análisis sintáctico se compone de cuatro capas claramente delimitadas que se describen a continuación.

4.2.1 Definición de la gramática

La gramática de HULK se especifica en dos formas equivalentes:

- un fichero `src/parser/grammar/hulk.grammar` en notación BNF extendida, legible para humanos y documentado con comentarios;
- una estructura `Vec<Production>` en `hulk_grammar.rs` construida programáticamente, que es la versión efectivamente usada por el generador de tablas.

4.2.2 Cálculo de FIRST y FOLLOW

Los conjuntos $\text{FIRST}(\alpha)$ y $\text{FOLLOW}(A)$ se calculan mediante el algoritmo clásico de punto fijo. Recordamos las definiciones:

DEFINICIÓN: CONJUNTOS FIRST Y FOLLOW

Sea $G = (N, T, P, S)$ una gramática libre de contexto, $A \in N$ un no-terminal y $\alpha \in (N \cup T)^*$ una forma sentencial.

$$\begin{aligned}\text{FIRST}(\alpha) &= \{a \in T \mid \alpha \Rightarrow^* a\beta\} \cup \{\varepsilon \mid \alpha \Rightarrow^* \varepsilon\} \\ \text{FOLLOW}(A) &= \{a \in T \cup \{\$ \} \mid S \Rightarrow^* \alpha A a \beta\}\end{aligned}$$

El algoritmo de punto fijo itera sobre las producciones actualizando los conjuntos hasta que en una iteración completa no cambia ningún conjunto.

4.2.3 Construcción de la tabla LALR

La construcción sigue el algoritmo estándar [1, 14] y se realiza en tres pasos. Primero se construye la colección canónica de *items* LR(1) mediante un BFS desde el *item* inicial $[\text{Start} \rightarrow \bullet \text{Program } \$, \$]$ tomando clausuras y aplicando $\text{GOTO}(I, X)$ para cada par estado-símbolo. Para la gramática completa de HULK este paso produce del orden de 500–600 estados LR(1). Segundo, se aplica el paso característico del algoritmo LALR(1) [3]: dos estados con idéntico *núcleo* (igual conjunto de pares producción-punto, ignorando *lookaheads*) se funden en uno solo, uniando sus conjuntos de *lookaheads*. La fusión reduce el espacio de estados a aproximadamente **263** estados, una reducción de cerca del 55 %. Tercero, se rellenan las tablas $\text{ACTION}[s, t]$ y $\text{GOTO}[s, N]$ recorriendo los *items* de cada estado fusionado y se registran los conflictos detectados para su inspección posterior.

TABLA 4.1. Estadísticas del autómata de parseo construido para HULK.

MÉTRICA	VALOR
Producciones declaradas	~ 90
Estados LR(1) antes de fusión	500–600
Estados LALR(1) tras fusión por núcleo	263
Entradas no triviales en ACTION	~1 500
Entradas no triviales en GOTO	~800
Conflictos <i>shift/reduce</i> sin resolver	0
Conflictos <i>shift/reduce</i> resueltos	1

El único conflicto de la gramática se sitúa sobre el terminal $|$ y nace de la interacción entre el operador OR lógico y el separador de las expresiones generadoras introducidas por la extensión (Capítulo 8). Cuando el motor de parseo se encuentra en un estado con la pila de la forma $[[\text{Expr}]]$ y el siguiente terminal es $|$, el *lookahead* estándar de un símbolo no basta: $[\text{Expr} \mid \dots]$ puede continuar tanto en $[\text{Expr} \mid \text{Expr}]$ —un vector explícito cuyo primer elemento es un OR lógico— como en $[\text{Expr} \mid \text{Id in Expr}]$ —un generador—. El constructor de la tabla resuelve este caso particular introduciendo un *lookahead* adicional de dos tokens: si tras el $|$ viene un IDENTIFIER seguido de *in*, se opta por la regla del generador; en cualquier

otro caso, por la regla de OR. La resolución se realiza en el momento de la construcción de la tabla, no en tiempo de parseo, por lo que el motor sigue siendo estrictamente LALR(1) en ejecución.

4.2.4 Motor de parseo

El motor implementa el algoritmo shift-reduce sobre las tablas precomputadas. Mantiene una pila de estados y una pila de valores semánticos en paralelo. En cada reducción se invoca una acción semántica que combina los valores semánticos del lado derecho de la producción en un nodo del AST.

4.3 Jerarquía de precedencias

La especificación oficial de HULK no establece la precedencia de operadores de forma explícita. La gramática implementada codifica la precedencia estandar matemática directamente en la jerarquía de no-terminales, evitando la necesidad de declaraciones de precedencia separadas. El Listado 4.1 muestra el esquema.

```
Expr      -> Assignment
Assignment -> OrExpr (':=' Assignment)?
OrExpr    -> AndExpr ('|' AndExpr)*
AndExpr   -> NotExpr ('&' NotExpr)*
NotExpr    -> '!' NotExpr | Equality
Equality   -> Comparison (('==' | '!=') Comparison)*
Comparison -> Concat (('<' | '>' | '<=' | '>=') Concat)*
Concat     -> Additive (('@' | '@@') Additive)*
Additive   -> Term (('+' | '-') Term)*
Term       -> Power (('*' | '/' | '%') Power)*
Power      -> Unary ('~' Power)?      (* asociatividad derecha *)
Unary      -> '-' Unary | Postfix
Postfix    -> Atom ('.' id '(' args ')' | '.' id | '[' Expr ']')*
Atom       -> literal | id | '(' Expr ')' | New | If | Let | ...
```

LISTADO 4.1. Codificación de precedencias en la gramática.

La asociatividad derecha de la potencia se obtiene mediante la recursión a la derecha en la regla Power: la expresión 2^3^2 se parsea como $2^{(3^2)} = 2^9 = 512$, no como $(2^3)^2 = 64$.

4.4 El árbol sintáctico abstracto

El AST está definido en `src/parser/ast/` mediante enumerados RUST con datos asociados. El nodo raíz es `Program`, que contiene una lista de declaraciones (`Decl`) y una expresión de entrada (`Expr`).

```
1 pub struct Program {
2     pub declarations: Vec<Decl>,
3     pub entry:      Expr,
4 }
5
6 pub struct Expr {
7     pub kind: ExprKind,
8     pub id:   u32,      // identificador único para la tabla de tipos
```

```

9      pub span: Span,      // línea y columna para mensajes de error
10     }
11
12     pub enum ExprKind {
13         Literal(Literal),
14         Identifier { name: String },
15         Binary(BinaryExpr),      Unary(UnaryExpr),
16         Postfix(PostfixExpr),    Assign(AssignExpr),
17         Let(LetExpr),            Block(BlockExpr),
18         If(IfExpr),              While(WhileExpr),
19         For(ForExpr),            Call(CallExpr),
20         MethodCall(MethodCallExpr),
21         Access(AccessExpr),      Index(IndexExpr),
22         New(NewExpr),            Base,
23         Is { expr: Box<Expr>, type_name: TypeName },
24         As { expr: Box<Expr>, type_name: TypeName },
25         Vector(VectorExpr),      // <-- extensión propia (cap. 8)
26     }

```

LISTADO 4.2. Estructura central del AST.

Cada Expr lleva un identificador único `id` que se utiliza como clave en la tabla `expr_types: HashMap<u32, HulkType>` que el analizador semántico construye. El generador de código consulta esta tabla para conocer el tipo inferido de cada subexpresión sin tener que reinferir.

5

Análisis semántico

CONSTRUIDO el AST, la siguiente fase verifica que el programa tenga sentido a nivel de tipos, alcances y referencias. Esta fase es responsable de detectar errores como uso de variables no declaradas, asignaciones entre tipos incompatibles, llamadas a funciones con número o tipo incorrecto de argumentos, y violaciones de la conformidad estructural a protocolos.

5.1 Estrategia en dos pasadas

El analizador semántico opera en dos pasadas claramente diferenciadas sobre la lista de declaraciones del programa.

Pasada 1 — Recolección de declaraciones. Todas las firmas de funciones, los miembros de tipos (atributos y métodos) y los cuerpos de protocolos se registran en la estructura `TypeHierarchy`. Los miembros sin anotación de tipo reciben un marcador `HulkType::Unknown` que sera resuelto en la segunda pasada. Esta pasada es indispensable para soportar **recursión mutua** entre funciones y tipos: en el momento de verificar el cuerpo de `foo`, la firma de `bar` ya esta disponible aunque `bar` este declarada después.

Pasada 2 — Verificación de cuerpos y propagación de tipos. Cada cuerpo de función, cada inicializador de atributo y cada cuerpo de método se verifica. El tipo inferido del cuerpo se compara con la anotación declarada si existe; si no existe, el tipo inferido se **escribe de vuelta** en la `TypeHierarchy` reemplazando el marcador `Unknown`.

Esta escritura es *crítica* para la fase de generación de código. Si un atributo `val = start` no tiene anotación de tipo, la pasada 1 almacena `Unknown`; la pasada 2 infiere `Number` a partir del inicializador; la escritura de vuelta hace que `Number` este disponible para el generador de código. Sin esta escritura, el codegen no podria determinar el tipo LLVM correcto del campo.

```
1 // Después de inferir el tipo del campo en la pasada 2:
2 if let Some(type_info) = self.types.types.get_mut(type_name) {
3     if let Some(attr) = type_info.attributes.get_mut(field_name) {
4         if *attr == HulkType::Unknown {
5             *attr = inferred_type.clone();
6         }
7     }
8 }
```

LISTADO 5.1. Propagación del tipo inferido en la pasada semántica.

5.2 Sistema de tipos

El sistema de tipos es **nominal** (la identidad de los tipos depende de su nombre y no de su estructura) con **subtype polymorphism** [6].

5.2.1 Categorías de tipo

- **Primitivos:** Number (IEEE-754 *double* de 64 bits), Boolean (un bit), String (puntero a cadena C *null*-terminada), Null (tipo singleton).
- **Definidos por el usuario:** introducidos mediante `type` con herencia simple opcional. Todos heredan transitivamente de `Object`.
- **Protocolos:** tipos de interfaz que listan firmas de métodos. La conformidad es **estructural**: un tipo conforma a un protocolo si declara todos sus métodos con firmas compatibles, sin necesidad de declarar la relación explícitamente.
- **Vectores:** el tipo `Vector<T>` para colecciones homogéneas de elementos de tipo `T` (vease Capítulo 8).
- **Tipos auxiliares:** `Unknown` (marcador en la pasada 1), `Never` (tipo de error para evitar cascadas de errores falsos).

5.2.2 Relación de conformidad

La relación de *conformidad* $\preceq$ entre tipos formaliza el subtipado. Se define inductivamente y opera tanto sobre la jerarquía nominal (tipos definidos por el usuario) como sobre la jerarquía estructural (protocolos).

DEFINICIÓN: CONFORMIDAD DE TIPOS

Sean A, B tipos del lenguaje HULK. Se dice que A *conforma a* B , notación $A \preceq B$, si y sólo si se cumple alguna de las siguientes condiciones:

$A = B$	(reflexividad)
A hereda transitivamente de B	(subtipado nominal)
B es protocolo y A lo implementa	(conformidad estructural)
$A = \text{Null}$ y B es nominal o <code>Object</code>	(nulabilidad)
$A = \text{Vec}\langle S \rangle$, $B = \text{Vec}\langle T \rangle$, $S \preceq T$	(covarianza de vectores)
$B = \text{Object}$	(raíz universal)

La conformidad estructural a un protocolo P no se reduce a una verificación puramente sintáctica de presencia de métodos: cada firma debe ser *compatible* con la del protocolo, donde compatibilidad significa **covarianza en el tipo de retorno y contravarianza en los parámetros**. Estas reglas son las clásicas para preservar la seguridad de la sustitución en presencia de subtipado, debidas originalmente a Liskov y Wing [6].

DEFINICIÓN: COMPATIBILIDAD DE FIRMAS CON VARIANZA

Sean $\sigma_A = (\bar{p}_A) \rightarrow r_A$ la firma del método declarado en el tipo candidato y $\sigma_P =$

$(\bar{p}_P) \rightarrow r_P$ la firma exigida por el protocolo. σ_A es compatible con σ_P si y sólo si:

$$\begin{array}{ll} |\bar{p}_A| = |\bar{p}_P| & \text{(aridad)} \\ r_A \preceq r_P & \text{(retorno covariante)} \\ \forall i. p_P^{(i)} \preceq p_A^{(i)} & \text{(parámetros contravariantes)} \end{array}$$

Estas reglas se implementan en el método `signatures_protocol_compatible` de la jerarquía de tipos: la conformidad se decide recorriendo los métodos del protocolo (y los del protocolo padre, si lo tiene) y delegando en la relación `conforms` para cada par de tipos. La búsqueda del método sube por la cadena de herencia del candidato, de modo que un método heredado satisface igual de bien la firma exigida que uno propio. El verificador atajará la búsqueda en cuanto detecte que el padre del candidato ya conforma al protocolo, evitando reexaminar firmas heredadas.

5.2.3 Unificación de tipos en condicionales: el ancestro común más cercano

Una situación recurrente en HULK es la determinación del tipo de una expresión `if-elif-else` cuyas ramas devuelven valores de tipos relacionados pero no idénticos. La regla aplicada es la habitual en lenguajes con subtipado: el tipo de la expresión es el *ancestro común más cercano* (*lowest common ancestor*, *lca*) de los tipos de todas las ramas.

DEFINICIÓN: ANCESTRO COMÚN MÁS CERCANO

$\text{lca}(A, B)$ se define como el tipo de menor profundidad en la jerarquía de herencia que satisface $A \preceq T \wedge B \preceq T$. Para n ramas se define recursivamente: $\text{lca}(T_1, T_2, \dots, T_n) = \text{lca}(\text{lca}(T_1, T_2), T_3, \dots, T_n)$.

El algoritmo implementado es directo: se recolecta el conjunto $\mathcal{A}(A)$ de todos los ancestros de A (incluido A) subiendo por la cadena de `TypeInfo.parent`, y se asciende desde B hasta encontrar el primer tipo que pertenece a $\mathcal{A}(A)$. La existencia de tal tipo está garantizada porque todos los tipos heredan transitivamente de `Object`, que actúa como elemento maximal del orden.

5.3 Funciones y constantes predefinidas

El compilador pre-registra el siguiente conjunto de elementos accesibles desde cualquier alcance, sin necesidad de importación:

TABLA 5.1. Funciones y constantes predefinidas del runtime HULK.

NOMBRE	DESCRIPCIÓN	TIPO
<code>print(x)</code>	Imprime el valor en stdout	<code>Object</code> $\rightarrow$ <code>Null</code>
<code>sqrt(x)</code>	Raíz cuadrada	<code>Number</code> $\rightarrow$ <code>Number</code>
<code>sin(x)</code>	Seno (radianes)	<code>Number</code> $\rightarrow$ <code>Number</code>
<code>cos(x)</code>	Coseno (radianes)	<code>Number</code> $\rightarrow$ <code>Number</code>
<code>exp(x)</code>	Exponencial e^x	<code>Number</code> $\rightarrow$ <code>Number</code>
<code>log(b, x)</code>	Logaritmo en base b	<code>(Number, Number)</code> $\rightarrow$ <code>Number</code>
<code>rand()</code>	Aleatorio uniforme en $[0, 1)$	<code>Number</code>
<code>range(s, e)</code>	Rango iterable $[s, e)$	<code>(Number, Number)</code> $\rightarrow$ <code>Range</code>
<code>PI, E</code>	Constantes matemáticas	<code>Number</code>
<code>true, false</code>	Literales booleanos	<code>Boolean</code>

5.4 Reporte de errores semánticos

Los errores semánticos se acumulan en un vector `Vec<SemanticError>` y se reportan todos juntos al final del análisis. Esta estrategia evita la frustración del programador que debe corregir un error a la vez. Cada `SemanticError` incluye el `Span` de la expresión o declaración causante, lo que produce mensajes con posición precisa:

```
! (12,5) SEMANTIC: type 'Cat' does not implement method 'speak' of
    protocol 'Animal'
! (18,3) SEMANTIC: cannot assign value of type 'String' to variable of
    type 'Number'
! (24,9) SEMANTIC: function 'foo' expects 2 arguments, 3 given
```


6

Generación de código

COMPLETADO el análisis semántico, el compilador dispone de un AST anotado con tipos y de una jerarquía de tipos verificada. La fase de generación de código traduce este AST en un modulo de LLVM Intermediate Representation, listo para ser optimizado y compilado a código máquina nativo.

6.1 LLVM IR mediante *inkwell*

LLVM [4] es la infraestructura de compilación mas utilizada del mundo. Provee una representación intermedia tipada y de bajo nivel, un conjunto extensible de *passes* de optimización, y *backends* para una variedad de arquitecturas (x86-64, ARM, RISC-V, WebAssembly, etc.).

La biblioteca *inkwell* es un *wrapper* RUST de la API C de LLVM que aprovecha el sistema de tipos del lenguaje para impedir construcciones invalidas en tiempo de compilación. Por ejemplo, los *builder* de instrucciones rechazan operaciones sobre bloques básicos ya terminados, y los valores estan parametrizados por el Context de LLVM para impedir mezclas entre contextos distintos.

El estado de la generación de código se centraliza en `CodegenContext`, una estructura que agrupa:

- `context`, `module`, `builder`: los tres objetos centrales de la API LLVM.
- `symbols`: pila de alcances para variables locales.
- `type_registry`: layouts de structs LLVM y vtables por tipo HULK.
- `type_hierarchy`: jerarquía de tipos del analizador semántico, accesible para resolución de métodos en herencia.
- `expr_types`: tipos anotados por el verificador, consultados durante la generación.
- `current_function`, `self_ptr`, `current_type_name`: contexto de la función o método en compilación.

6.2 Representación de valores: el enum `CgValue`

Dado que HULK tiene tipos heterogeneos (`Number` es escalar, `String` y los objetos son punteros), no existe un único tipo LLVM universal. La representación uniforme se obtiene mediante un enum RUST que transporta el valor LLVM junto con su categoría de tipo HULK:

```
1 pub enum CgValue<'ctx> {  
2     Number(FloatValue<'ctx>), // LLVM: double  
3     Bool(IntValue<'ctx>), // LLVM: i1
```

```

4   Str(PointerValue<'ctx>),      // LLVM: ptr a cadena C
5   Object(PointerValue<'ctx>),   // LLVM: ptr al struct del tipo
6   Vector(PointerValue<'ctx>),   // LLVM: ptr al buffer en heap
7   Null,                        // puntero nulo
8   Void,                        // sin valor (efectos de lado)
9 }

```

LISTADO 6.1. Representación de valores en codegen.

Las coerciones entre variantes se realizan en `coerce_arg`, que inserta las instrucciones LLVM necesarias: `Bool` a `Number` usa `uitofp`, cualquier tipo primitivo a `ptr` para paso a `print(x: Object)` se empaqueta mediante un `alloca + store`.

6.3 Representación de objetos en memoria

Todo tipo definido por el usuario se representa como un `struct` LLVM con una distribución de campos uniforme. Los dos primeros campos son metadatos comunes a todos los objetos; los siguientes son los campos declarados por el usuario en orden *padre-primero*.

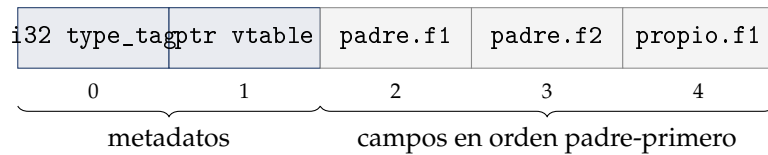


FIGURA 6.1. Layout en memoria de un objeto HULK que hereda de un padre con dos campos.

El orden *padre primero* es deliberado: garantiza que un puntero a un objeto del tipo `Dog` (que hereda de `Animal`) sea **bit-compatible** con un puntero a `Animal` respecto a los campos heredados. Esto evita el costo de reinterpretación de bits o de tablas de desplazamiento que serían necesarias con otros layouts.

6.4 Tablas de métodos virtuales (vtables)

El polimorfismo dinámico de HULK se implementa mediante *vtables*, la técnica estandar utilizada por C++, Java (compilado JIT) y otros lenguajes con herencia y polimorfismo.

Para cada tipo se crea una **vtable global** (un global LLVM) que contiene punteros a los métodos del tipo, en un orden fijo: primero los métodos heredados del padre, luego los nuevos. Cuando un tipo hace *override* de un método del padre, la posición en la vtable se conserva pero el puntero se reemplaza por el del nuevo método.

```

1 %VTable_Dog = type { ptr, ptr }
2
3 @vtable_Dog = global %VTable_Dog {
4     ptr @__hulk_method_Dog_speak,      ; override
5     ptr @__hulk_method_Animal_move    ; heredado del padre
6 }

```

LISTADO 6.2. Vtable global para un tipo `Dog` que redefine `speak` y hereda `move`.

El despacho de un método `x.speak()` sobre un valor `x` de tipo estático `Animal` se compila como un acceso indirecto en tres pasos: cargar el puntero a `vtable` desde el campo 1 del objeto, indexar al slot precalculado para el método `speak`, e invocar el puntero a función con la convención de llamada habitual (`self` en la primera posición, argumentos a continuación).

6.4.1 Despacho de métodos a través de un protocolo

El esquema de `vtables` es suficiente cuando el tipo estático del receptor es un tipo nominal: en ese caso, el orden de los `slots` es el mismo en todas las clases de la jerarquía y la posición del método se calcula en tiempo de compilación. Sin embargo, cuando el tipo estático es un protocolo `P`, los `slots` pueden diferir entre tipos conformantes: si `A` declara `{m,n}` y `B` declara `{n,m}`, la posición de `m` en cada `vtable` es distinta, y un acceso por índice fijo daría el método equivocado.

La estrategia elegida en este compilador resuelve esa indeterminación con un *switch* sobre el `type_tag` leído en tiempo de ejecución, generando una rama por cada tipo conforme conocido y, en cada rama, una llamada *directa* al método correspondiente. La construcción es local a la expresión de despacho —no requiere tablas globales adicionales— y se beneficia de la optimización clásica que LLVM aplica a los `switch`: tras `simplifycfg` se transforma en una *jump table*, lo que asintóticamente tiene el mismo coste que un acceso a `vtable`. El bloque `default` del `switch` se marca `unreachable` porque el verificador de tipos garantiza que el receptor realmente conforma al protocolo:

```

1  %tag = load i32, ptr %b
2
3  switch i32 %tag, label %proto_unreachable [
4      i32 3, label %proto_case_MBox
5      i32 5, label %proto_case_Cylinder
6  ]
7
8  proto_case_MBox:
9      %r1 = call f64 @__hulk_method_MBox_measure(ptr %b)
10     br label %proto_merge
11
12  proto_case_Cylinder:
13      %r2 = call f64 @__hulk_method_Cylinder_measure(ptr %b)
14      br label %proto_merge
15
16  proto_unreachable:
17      unreachable ; semánticamente inalcanzable
18
19  proto_merge:
20      %result = phi f64 [ %r1, %proto_case_MBox ], [ %r2, %proto_case_Cylinder
    ]

```

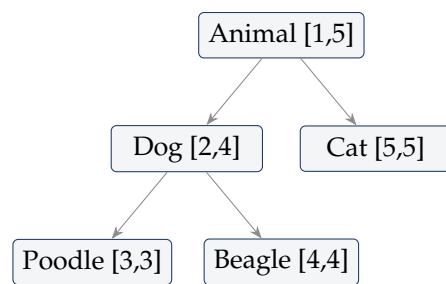
LISTADO 6.3. Despacho a través de un protocolo `Measurable` con dos tipos conformes.

La decisión entre despacho por `vtable` (para tipos nominales) y despacho por *switch* (para protocolos) se toma en `method_dispatch` del módulo `codegen::context` consultando la categoría sintáctica del tipo estático del receptor.

6.5 Etiquetas de tipo en orden DFS

El operador `is` (`x is Dog`) es una operación frecuente en código OOP con herencia, y se utiliza también internamente en la implementación del operador `as` (*downcast*). Implementarlo eficientemente requiere un invariante no trivial sobre la asignación de etiquetas de tipo.

Asignación DFS pre-orden. Durante la compilación, los tipos reciben **etiquetas enteras en orden de recorrido en profundidad de la jerarquía de herencia**, en pre-orden. El tipo padre recibe su etiqueta antes que sus hijos. Como consecuencia, **todos los subtipos directos e indirectos de un tipo T forman un intervalo contiguo** $[\min_tag_T, \max_tag_T]$ en el espacio de etiquetas.



poodle is Dog: $2 \leq 3 \leq 4$ ✓
 poodle is Animal: $1 \leq 3 \leq 5$ ✓
 cat is Dog: $2 \leq 5 \leq 4$ ✗

FIGURA 6.2. Asignación DFS pre-orden de etiquetas de tipo. El test `is` se reduce a una verificación de pertenencia a un intervalo, computable en tiempo constante.

Implementación del operador `is`. Gracias a esta asignación, el código LLVM generado para `x is Dog` es simplemente:

```

%tag = load i32, ptr %x
%ge  = icmp uge i32 %tag, 2      ; min_tag(Dog)
%le  = icmp ule i32 %tag, 4      ; max_tag(Dog)
%res = and i1 %ge, %le
  
```

Esto es $O(1)$ en tiempo de ejecución, sin consultar ninguna tabla ni recorrer la jerarquía. La técnica esta documentada en implementaciones eficientes de RTTI en C++ [5].

6.6 El operador `as`: *downcast* con verificación

El operador `x as T` realiza un *downcast*: convierte un valor de tipo base (por ejemplo `Animal`) al tipo derivado `T` (por ejemplo `Dog`). En HULK, esto debe ser *seguro*: si el objeto realmente no es de tipo `T`, la ejecución debe abortar con un mensaje de error en lugar de continuar con un puntero malformado.

El código generado primero verifica el rango de etiquetas (igual que en `is`). Si la verificación falla, llama a la función de *runtime* `hulk_type_error`, que imprime el mensaje de error y llama a `abort(3)`. Si la verificación tiene éxito, el puntero original se utiliza directamente: no se requiere reinterpretación de bits porque el layout es compatible por la organización padre-primero.

6.7 La palabra clave `base()`

Dentro de un método, `base(args)` invoca la implementación del padre. A diferencia del despacho virtual — que cargaría la `vtable` y podría producir una llamada recursiva si el padre llama de vuelta al método del hijo —, `base()` es una **llamada directa estática**: el nombre de la función del padre se conoce en tiempo de compilación.

El código generador busca recursivamente en la jerarquía hasta encontrar el tipo que realmente implementa el método (no necesariamente el padre inmediato, si varios niveles intermedios solo lo heredan), y emite una llamada directa por nombre.

6.8 Pipeline de optimización

El IR generado por el codegen es deliberadamente sencillo y abundante en *allocas*: cada variable local se modela como una secuencia `alloca + store + load`. Esta forma facilita la generación correcta pero es ineficiente sin optimización.

Antes de la emisión de código objeto se aplica el *pipeline* del nuevo *pass manager* de LLVM 17 con la siguiente secuencia:

TABLA 6.1. Passes de optimización aplicados al IR.

PASS	EFFECTO
<code>mem2reg</code>	Promueve patrones <code>alloca/store/load</code> a registros SSA virtuales. Es el habilitador fundamental de todas las optimizaciones subsiguientes.
<code>instcombine</code>	Plegado de constantes, simplificación algebraica, eliminación de operaciones redundantes (por ejemplo, $x \oplus 0 = x$, $x \cdot 1 = x$).
<code>reassociate</code>	Reordena expresiones asociativas para maximizar la oportunidad de plegado por <code>instcombine</code> .
<code>simplifycfg</code>	Elimina bloques básicos muertos, fusiona bloques redundantes, simplifica el grafo de flujo de control.

Esta combinación produce un IR significativamente mas compacto y próximo al código generado por compiladores maduros como `gcc -O2`.

7

Runtime y compilación final

EL código LLVM producido por el *frontend* no es autosuficiente: depende de un conjunto de funciones de soporte para operaciones como impresión, formato de números, allocación de cadenas y manejo de colecciones. Estas funciones forman la *biblioteca de runtime*, escrita en C y enlazada con cada ejecutable producido.

7.1 La biblioteca de runtime en C

El *runtime* se compila a un archivo estático `hulk_runtime.a` desde `runtime/hulk_runtime.c`. Provee las siguientes funciones, todas con enlace C y nombres no decorados para que sean accesibles desde el código generado.

TABLA 7.1. Funciones de la biblioteca de runtime.

FUNCIÓN	DESCRIPCIÓN
<code>hulk_print(ptr)</code>	Imprime cadena + <i>newline</i> a stdout.
<code>hulk_str_from_number(d)</code>	Convierte double a C string; formatea enteros sin parte decimal.
<code>hulk_str_concat(a,b)</code>	Concatenación para el operador @.
<code>hulk_str_concat_space(a,b)</code>	Concatenación con espacio para @@.
<code>hulk_str_eq(a,b)</code>	Igualdad de cadenas por valor.
<code>hulk_str_size(s)</code>	Longitud en bytes de la cadena.
<code>hulk_vec_alloc(n,sz)</code>	Alloca buffer: header (8 bytes) + $n \times 8$ bytes de elementos.
<code>hulk_vec_get(p,i,sz)</code>	Devuelve puntero al elemento <i>i</i> con verificación de límites.
<code>hulk_vec_size(p)</code>	Lee el header y devuelve el conteo.
<code>hulk_range_alloc(s,e)</code>	Crea iterador para el rango $[s,e)$.
<code>hulk_range_next(p)</code>	Avanza el iterador; devuelve true si hay mas elementos.
<code>hulk_range_current(p)</code>	Valor actual del iterador.
<code>hulk_rand()</code>	Numero aleatorio uniforme en $[0,1)$.
<code>hulk_type_error(p)</code>	Imprime el mensaje y aborta (as fallido).

7.2 Layout en memoria de vectores y rangos

Las dos estructuras compuestas del *runtime*, **vectores** y **rangos**, tienen layouts compactos diseñados para minimizar el número de *allocs* y maximizar la localidad de acceso.

8

Extensión del lenguaje: vectores con expresiones generadoras

ESTE capítulo presenta la extensión central del lenguaje implementada en este compilador: un **sistema de vectores con expresiones generadoras**, estáticamente tipado, integrado con el sistema de protocolos del lenguaje y con verificación segura de índices en tiempo de ejecución. La extensión modifica simultáneamente la sintaxis —añade literales de vector, una notación generadora inspirada en la *set-builder notation* matemática, y un operador de indexación—, el sistema de tipos —introduce un constructor de tipos paramétrico `Vector<T>` con regla de varianza definida—, la semántica operacional —introduce reglas de evaluación con un mecanismo de aborto controlado para índices inválidos— y el *runtime* del lenguaje —añade primitivas de asignación, acceso y medida de tamaño con verificación de límites.

8.1 Motivación

En su núcleo básico, HULK no incluye estructuras de datos compuestas: las únicas formas no escalares disponibles son los rangos producidos por la función `range(s, e)` (que tienen una semántica puramente iterable, sin acceso aleatorio) y los tipos definidos por el usuario. Para escribir algoritmos elementales que operen sobre colecciones —calcular el promedio de una lista de números, aplicar una función a cada elemento, filtrar los valores que cumplen una condición— el programador debe declarar un tipo propio que encapsule la lógica de tamaño, acceso e iteración. Esta declaración no es difícil pero es sistemáticamente verbosa, en un lenguaje cuya virtud central es precisamente la concisión expresiva.

La propia especificación del lenguaje reconoce esta carencia y bosqueja, en una sección dedicada de su apéndice gramatical, un sistema de vectores como extensión natural. La extensión presentada en este capítulo toma esa propuesta como punto de partida y la enriquece en cuatro sentidos:

- agrega **indexación** con la sintaxis `v[i]` y verificación de límites en *runtime*;
- integra los vectores con el **protocolo** `Iterable`, permitiendo usarlos en `for` igual que los rangos;
- define el **tipo** `Vector<T>` en el sistema de tipos con inferencia bidireccional;
- proporciona un **runtime con bounds-checking** que aborta de forma controlada en accesos inválidos.

8.2 Definición formal de la sintaxis

La extensión agrega tres construcciones sintácticas nuevas a la gramática de HULK: el vector explícito, la expresión generadora y la indexación.

```
(* Atom es extendido con vectores literales *)
Atom      -> ... | VectorLiteral

VectorLiteral -> '[' ']'          (* vector
    vacío *)
              | '[' ArgListNonEmpty ']'          (* explícito
    *)
              | '[' Expr '|' IDENTIFIER 'in' Expr ']'          (* generador
    *)

(* Postfix es extendido con indexación *)
Postfix     -> Atom (... | '[' Expr ']' ) *      (*
    indexación *)
```

LISTADO 8.1. Nuevas producciones gramaticales para la extensión.

8.2.1 Vectores explícitos

Un **vector explícito** se construye listando sus elementos entre corchetes, separados por comas. El tipo del vector se infiere a partir del tipo de sus elementos.

```
1 let v      = [1, 2, 3, 4, 5]          in print(v[2]);    // 3
2 let names = ["Alice", "Bob", "Carol"] in print(names[0]);
3 let empty = []                      in print(size(empty)); // 0
```

LISTADO 8.2. Vectores explícitos.

8.2.2 Expresiones generadoras

La sintaxis central de la extensión es la **expresión generadora**, que crea un nuevo vector aplicando una expresión a cada elemento de un iterable.

```
1 // Cuadrados de 0 a 4
2 let squares = [x * x | x in range(0, 5)] in
3 print(squares[3]);                      // 9
4
5 // Transformación de otro vector
6 let v      = [1, 2, 3]                  in
7 let doubled = [x * 2 | x in v]           in
8 print(doubled[1]);                      // 4
9
10 // El cuerpo puede usar variables del entorno
11 let factor = 10 in
12 let scaled = [x * factor | x in range(1, 4)] in
13 print(scaled[2]);                      // 30
```

LISTADO 8.3. Expresiones generadoras.

La sintaxis es `[body | var in iterable]`, donde:

- *body* es cualquier expresión HULK que puede usar *var* y variables del entorno circundante;
- *var* es la variable de iteración, local al cuerpo;
- *iterable* es un *Range* o un *Vector<T>* de cualquier tipo;
- el separador *|* coincide con el del operador OR lógico; la ambigüedad se resuelve por contexto (vease el conflicto declarado en la gramática).

8.2.3 Indexación

Los elementos de un vector se acceden con la sintaxis $v[i]$, donde i es una expresión de tipo *Number*.

```

1 let v = [10, 20, 30]                                     in
2 let i = 1                                                in
3 v[i + 1] // equivalente a v[2] = 30

```

LISTADO 8.4. Indexación con expresión calculada.

El índice se convierte de *Number (double)* a entero en tiempo de compilación mediante *fptos.i*. El *runtime* verifica que el índice sea no-negativo y menor que el tamaño del vector; en caso contrario, aborta con un mensaje descriptivo.

8.3 Definición formal de la semántica

Para precisar el significado de las nuevas construcciones, se presentan las reglas de tipado y la semántica operacional en estilo natural.

8.3.1 Reglas de tipado

DEFINICIÓN: TIPOS DE LA EXTENSIÓN

Se introduce un nuevo constructor de tipos *Vector<T>* paramétrico en un parámetro T . La conformidad de vectores es **invariante** en el parámetro: $\text{Vector}\langle A \rangle \leq \text{Vector}\langle B \rangle \iff A = B$.

Las reglas de tipado para las nuevas expresiones son las siguientes, donde Γ es el ambiente de tipos y $\vdash$ denota el juicio de tipado:

$$\begin{array}{ll}
 \frac{\Gamma \vdash e_1 : T \quad \dots \quad \Gamma \vdash e_n : T}{\Gamma \vdash [e_1, \dots, e_n] : \text{Vector}\langle T \rangle} & \text{(vector explícito)} \\
 \frac{\Gamma \vdash e : \text{Range} \quad \Gamma, x : \text{Number} \vdash b : T}{\Gamma \vdash [b \mid x \text{ in } e] : \text{Vector}\langle T \rangle} & \text{(generador sobre rango)} \\
 \frac{\Gamma \vdash e : \text{Vector}\langle S \rangle \quad \Gamma, x : S \vdash b : T}{\Gamma \vdash [b \mid x \text{ in } e] : \text{Vector}\langle T \rangle} & \text{(generador sobre vector)} \\
 \frac{\Gamma \vdash e : \text{Vector}\langle T \rangle \quad \Gamma \vdash i : \text{Number}}{\Gamma \vdash e[i] : T} & \text{(indexación)}
 \end{array}$$

8.3.2 Semántica operacional de gran paso

La semántica de evaluación de los vectores y de los generadores adopta una estrategia **ansiosa** (*eager*): todos los elementos se calculan en orden de izquierda a derecha y se almacenan en un buffer recién asignado *antes* de que la expresión completa retorne. Esta elección está alineada con la semántica del resto de HULK —donde los argumentos de funciones, los lados de los operadores binarios y los inicializadores de `let` se evalúan de forma estricta— y se justifica con detalle en la Apartado 8.6.

Se presenta la semántica en estilo de *gran paso* (también llamada *natural* o *evaluativa*): el juicio $\sigma \vdash e \Downarrow v$, σ' se lee “en el entorno σ , la expresión e se evalúa al valor v produciendo el entorno σ' ”. El entorno σ asocia variables a valores, y $\sigma[x \mapsto v]$ denota la extensión que añade el binding $x \mapsto v$ ocultando cualquier binding previo.

$$\frac{\sigma \vdash e_1 \Downarrow v_1, \sigma_1 \quad \dots \quad \sigma_{n-1} \vdash e_n \Downarrow v_n, \sigma_n}{\sigma \vdash [e_1, \dots, e_n] \Downarrow \text{vec}(v_1, \dots, v_n), \sigma_n} \quad (\text{E-VecExp})$$

$$\frac{\sigma \vdash e \Downarrow \text{vec}(v_1, \dots, v_n), \sigma_0 \quad \forall i. \sigma_0[x \mapsto v_i] \vdash b \Downarrow w_i, \sigma_i}{\sigma \vdash [b \mid x \text{ in } e] \Downarrow \text{vec}(w_1, \dots, w_n), \sigma_n} \quad (\text{E-VecGenV})$$

$$\frac{\sigma \vdash e \Downarrow \text{range}(s, t), \sigma_0 \quad n = \max(0, \lfloor t - s \rfloor) \quad \forall i \in [0, n). \sigma_0[x \mapsto s + i] \vdash b \Downarrow w_i}{\sigma \vdash [b \mid x \text{ in } e] \Downarrow \text{vec}(w_0, \dots, w_{n-1}), \sigma_n} \quad (\text{E-VecGenR})$$

$$\frac{\sigma \vdash e \Downarrow \text{vec}(v_0, \dots, v_{n-1}), \sigma_0 \quad \sigma_0 \vdash i \Downarrow k, \sigma_1 \quad 0 \leq \lfloor k \rfloor < n}{\sigma \vdash e[i] \Downarrow v_{\lfloor k \rfloor}, \sigma_1} \quad (\text{E-IndexOk})$$

$$\frac{\sigma \vdash e \Downarrow \text{vec}(v_0, \dots, v_{n-1}), \sigma_0 \quad \sigma_0 \vdash i \Downarrow k, \sigma_1 \quad \neg(0 \leq \lfloor k \rfloor < n)}{\sigma \vdash e[i] \Downarrow \perp} \quad (\text{E-IndexErr})$$

La regla E-INDEXERR captura el comportamiento de aborto controlado del *runtime*: cuando el índice está fuera de rango, no se produce un valor sino una transición a un estado de error (notado $\perp$) que el *runtime* materializa imprimiendo un mensaje en `stderr` y llamando a `abort` (3). Este modelo de fallos sigue el principio de Hoare según el cual *una condición que no debería ocurrir no debe producir un valor arbitrario; debe detener la ejecución de la manera más visible posible* [15].

8.4 Realización en el código

8.4.1 Modificaciones al lexer

Ninguna. Los tokens `[`, `]`, `|`, `e in` ya existen en el lexer para otras construcciones. La extensión reutiliza el conjunto existente sin agregar terminales.

8.4.2 Modificaciones al parser

Se agregan las producciones del Listado 8.1. El único conflicto `shift/reduce` nuevo aparece en `[Expr | ...` porque el token `|` podría iniciar también el operador OR lógico de la regla `OrExpr`. El conflicto se resuelve con un *lookahead* de dos tokens: si tras `|` viene un `IDENTIFIER` seguido de `in`, se reduce hacia `VectorLiteral`; en cualquier otro caso, se desplaza hacia `OrExpr`.

El AST se extiende con dos variantes:

```

1 pub enum VectorExpr {
2     Explicit { elements: Vec<Expr>, span: Span },
3     Generator { body: Box<Expr>, var: String,
4                 iterable: Box<Expr>, span: Span },
5 }
6
7 pub struct IndexExpr {
8     pub collection: Box<Expr>,
9     pub index:      Box<Expr>,
10    pub span:        Span,
11 }

```

LISTADO 8.5. Nuevas variantes del AST para la extensión.

8.4.3 Modificaciones al semántico

El verificador de tipos implementa las reglas de la sección anterior: para un vector explícito, infiere el tipo del primer elemento y verifica que los demás sean compatibles; para una expresión generadora, evalúa el tipo del iterable y construye un ambiente extendido $\Gamma, x : T$ antes de tipar el cuerpo; para una indexación, verifica que la colección sea `Vector<T>` para algún `T` y que el índice sea `Number`.

8.4.4 Modificaciones al codegen

La generación de código para vectores explícitos es directa: una llamada a `hulk_vec_alloc` para reservar el buffer, seguida de una secuencia de `hulk_vec_get` (para obtener punteros a los slots) y `store` (para escribir los valores).

La generación para una expresión generadora es más elaborada y se ilustra en el Figura 8.1: requiere construir un bucle con cuatro bloques básicos (`gen_cond`, `gen_body`, `gen_end`, más el bloque de entrada).

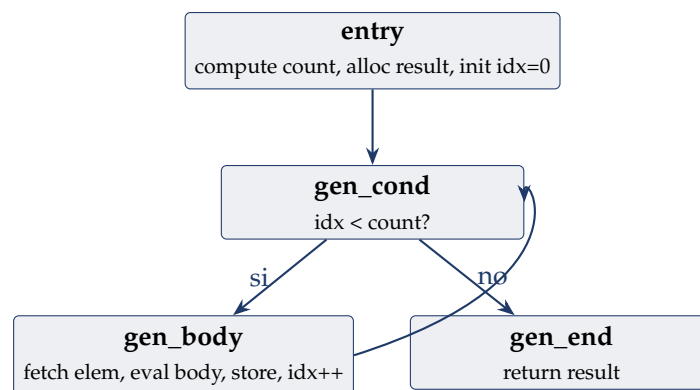


FIGURA 8.1. Grafo de flujo de control del código generado para una expresión generadora.

Una sutileza importante: el cálculo del `count` para un generador sobre un `Range` hace `count = floor(end - start)`. Sin precauciones, un rango con `start > end` produciría `count < 0`, lo que provocaría que la instrucción `fptosi` de LLVM generara un *poison value* y el *behavior undefined*. La extensión inserta un *clamp* a `[0, i32::MAX]`:

```

1 %diff    = fsub double %end, %start
2 %is_neg  = fcmp ult double %diff, 0.0
3 %nn      = select i1 %is_neg, double 0.0, double %diff
4 %hi      = fcmp ogt double %nn, 2147483647.0
5 %clamp   = select i1 %hi, double 2147483647.0, double %nn
6 %count   = fptosi double %clamp to i32

```

LISTADO 8.6. Clamp del count para evitar poison en fptosi.

8.5 Análisis comparativo con otros lenguajes

La construcción implementada esta emparentada con una familia de construcciones presentes en otros lenguajes bajo nombres como *list comprehension*, *for comprehension* o *generator expression*. A continuación se compara la extensión propuesta con las equivalentes mas relevantes.

8.5.1 Notación matemática de conjuntos

La inspiración histórica de todas estas construcciones es la **notación de construcción de conjuntos** (*set-builder notation*) introducida en la teoría axiomática de conjuntos a principios del siglo XX:

$$S = \{f(x) \mid x \in A \wedge P(x)\}.$$

Esta notación, leída “*el conjunto de los $f(x)$ tales que x pertenece a A y $P(x)$ se cumple*”, se para con una barra vertical la expresión generadora de los cuantificadores. La extensión propuesta toma directamente esta convención sintáctica: `[body | var in iterable]` es la traducción literal de la notación matemática al ASCII, con la barra vertical `|` como separador y el *keyword* `in` en sustitución del símbolo $\in$. Es importante notar que la notación matemática puede tener *cualquier* número de generadores y de predicados; la extensión que aquí se presenta admite, por simplicidad de la gramática LALR(1), exactamente un generador y ningún predicado. La omisión es deliberada (vease Apartado 8.6) y la pérdida de expresividad es moderada porque cualquier filtro puede emularse insertando un `if-else` en el cuerpo de la comprensión.

8.5.2 Comparación con Python

Python distingue entre **list comprehensions** (*eager*) y **generator expressions** (*lazy*):

```

1 squares = [x**2 for x in range(5)]      # list comp, eager
2 gen      = (x**2 for x in range(5))     # generator expr, lazy

```

LISTADO 8.7. Comprehensions y generators en Python.

La sintaxis de HULK `[body | var in iterable]` corresponde a las *list comprehensions* de Python tanto en comportamiento (*eager*, materializan toda la lista) como en semántica de alcance (`var` es local al cuerpo, no captura el contexto).

Diferencias relevantes:

- Python admite **filtros** (`if x > 0`) y **múltiples iteradores** (`for x in xs for y in ys`); la extensión de HULK se limita a un solo iterador sin filtro. Esta simplificación fue intencional: mantiene la gramática LALR(1) libre de ambigüedades difíciles y reduce la complejidad del codegen.
- Python es dinámicamente tipado y permite vectores heterogeneos; HULK es estáticamente tipado e impone homogeneidad: todos los elementos deben conformar al mismo tipo.

8.5.3 Comparación con Haskell

Haskell ofrece *list comprehensions* con una sintaxis aun mas cercana a la notación matemática:

```
1 squares = [x^2 | x <- [0..4]]
2 evens    = [x | x <- [1..100], even x]
```

LISTADO 8.8. List comprehensions en Haskell.

La evaluación en Haskell es **perezosa por defecto**: los elementos se computan solo cuando son demandados. Esto permite construir listas infinitas como `nats = [0..]`, donde los valores se calculan incrementalmente.

HULK adopta evaluación **ansiosa**. Las razones de esta decisión se exponen en la Apartado 8.6. El precio que se paga es la imposibilidad de manejar colecciones infinitas; el beneficio es un modelo de memoria y de ejecución mucho mas simple, compatible con el resto del lenguaje (que también es eager).

8.5.4 Comparación con Scala

Scala ofrece la construcción `for...yield`, una azucar sintáctica sobre `flatMap` y `map`:

```
1 val squares = for (x <- 0 until 5) yield x * x
2 val pairs   = for {
3   x <- 1 to 3
4   y <- 1 to x
5 } yield (x, y)
```

LISTADO 8.9. For-comprehension en Scala.

Scala es *eager* para colecciones concretas como `Vector` y *lazy* para vistas `.view`. La sintaxis es mas verbosa pero soporta multiples generadores y guardas. La elección de HULK de exigir un solo generador minimiza la complejidad sintáctica al precio de la expresividad.

8.5.5 Comparación con C++20 ranges

C++20 introdujo el sistema de **ranges y views** con evaluación perezosa y composición por *pipe*:

```
1 auto squares = std::views::iota(0, 5)
2               | std::views::transform([](int x){ return x*x; });
3 std::vector<int> v(squares.begin(), squares.end()); // materializa
```

LISTADO 8.10. Ranges y views en C++20.

C++ favorece evaluación perezosa con *materialización explícita*, lo que permite optimizar la composición de transformaciones evitando allocaciones intermedias. Para HULK — que no posee *iterators* genéricos ni un sistema de tipos con *lifetimes* — esta aproximación sería desproporcionadamente compleja.

8.5.6 Resumen comparativo

TABLA 8.1. Comparativa de comprehensions/generators entre lenguajes.

LENGUAJE	SINTAXIS	EVALUACIÓN	FILTROS	TIPOS
HULK	[e x in it]	Ansiosa	No	Estático inferido
Python	[e for x in it if c]	Ansiosa/Diferida	Sí	Dinámico
Haskell	[e x <- xs, c]	Diferida	Sí	Estático inferido
Scala	for x <- xs yield e	Ansiosa/Diferida	Sí	Estático inferido
C++20	xs transform(f)	Diferida	Sí	Estático explícito

8.6 Decisiones de diseño de la extensión

8.6.1 Por que evaluación ansiosa y no perezosa

Tres argumentos sostienen la decisión de evaluación ansiosa.

Simplicidad del modelo de memoria. Con evaluación ansiosa el tamaño del vector se conoce antes de la asignación, por lo que se requiere una sola llamada a `malloc` con la capacidad exacta. La evaluación perezosa requeriría *thunks* (cierres con estado) o continuaciones, estructuras adicionales que duplicarían la complejidad del *runtime* sin un beneficio significativo en el caso de uso típico.

Rendimiento predecible. Los compiladores de lenguajes perezosos como GHC para Haskell invierten esfuerzo considerable en evitar los *overheads* de la *thunkificación*: el análisis de demanda, el análisis de *strictness* y la deforestación son optimizaciones complejas dedicadas a este problema. En un compilador con fines educativos que ya delega la optimización en LLVM, la evaluación ansiosa produce IR simple y fácil de optimizar con los *passes* estandar.

Consistencia con el resto del lenguaje. HULK tiene efectos de lado (`print`, asignación destructiva). La evaluación perezosa en presencia de efectos produce comportamientos sorprendentes porque el orden de evaluación deja de coincidir con el orden textual. La evaluación ansiosa garantiza que los efectos se producen de izquierda a derecha, en consonancia con la semántica del resto de HULK.

8.6.2 Por que la barra simple | como separador

La barra vertical reproduce la notación matemática $\{f(x) \mid x \in S\}$ y coincide con la convención de Haskell `[f x | x <- xs]`. El hecho de que `|` sea también el operador OR lógico introduce un conflicto shift/reduce que se resuelve con un *lookahead* adicional. Alternativas consideradas:

- `[body for var in iterable]` — mas familiar para programadores Python, pero introduce el keyword `for` en un contexto distinto al de los bucles `for`, lo que dificulta el aprendizaje del lenguaje.
- `[body : var in iterable]` — colisiona con la notación de tipo `x : T` ya presente en la gramática de HULK.
- `[body || var in iterable]` — evita el conflicto pero rompe la correspondencia con la notación matemática.

8.6.3 Por que vectores inmutables (tamaño fijo)

Los vectores de HULK no tienen operaciones de `push` o `pop`: una vez creados, su tamaño no cambia. Las modificaciones afectan solo a los elementos. Esta decisión se justifica por:

1. **Simplicidad del runtime:** un buffer fijo con cabecera es trivial. Vectores dinámicos requerirían estrategias de redimensión (duplicar capacidad) que duplicarían la complejidad de la biblioteca de soporte.
2. **Semántica de expresión:** en HULK todo es una expresión. Un vector dinámico con mutación es un objeto con estado, lo que rompe la composición funcional que permite escribir `print([x*x | x in range(0,5)][3])`.
3. **Caso de uso del lenguaje:** HULK es un lenguaje educativo; sus programas típicos son cortos y operan sobre vectores de tamaño conocido.

9

Cobertura de la especificación

ESTE capítulo cuantifica el grado de cobertura del compilador respecto a la especificación oficial del lenguaje HULK. La especificación organiza el lenguaje en las secciones A.2 a A.14 de su apéndice gramatical: las secciones A.2 a A.8 forman el núcleo del lenguaje, mientras que las restantes describen extensiones opcionales que pueden incorporarse de manera incremental.

9.1 Tabla de cobertura por sección

TABLA 9.1. Cobertura de la especificación oficial de HULK por sección.

SEC.	CARACTERÍSTICA	OBL.	ESTADO
A.2.1	Expresiones aritméticas, precedencia	si	✓
A.2.2	Cadenas, concatenación, escapes	si	✓
A.2.3	Funciones matemáticas builtin	si	✓
A.2.4	Bloques de expresiones	si	✓
A.3	Funciones (inline y completas)	si	✓
A.4	Variables y let-in	si	✓
A.4.6	Asignación destructiva	si	✓
A.5	Condicionales if-elif-else	si	✓
A.6.1	Ciclo while	si	✓
A.6.2	Ciclo for	si	✓
A.7	Tipos, herencia, self, base()	si	✓
A.8.1	Anotaciones de tipo	si	✓
A.8.2	Conformidad <=	si	✓
A.8.3	Operador is	si	✓
A.8.4	Operador as (downcast)	si	✓
A.9	Inferencia de tipos (parcial)	no	○
A.10	Protocolos y conformidad estructural	no	✓
A.11	Iterables (Iterable/Range)	no	✓
A.12	Vectores y generadores (extensión)	no	✓
A.13	Funtores y lambdas	no	✗
A.14	Macros con pattern matching	no	✗
COBERTURA OBLIGATORIA		100 %	

9.2 Ejemplos representativos

El compilador es capaz de procesar programas como los siguientes, que ejercitan múltiples características a la vez.

9.2.1 Herencia y polimorfismo

```

1  type Animal(name: String) {
2      name = name;
3      speak(): String => "...";
4      greet(): String =>
5          "I am " @@ self.name @@ " and I say: " @@ self.speak();
6  }
7
8  type Dog(name: String) inherits Animal(name) {
9      speak(): String => "Woof!";
10 }
11
12 type Cat(name: String) inherits Animal(name) {
13     speak(): String => "Meow!";
14 }
15
16 let zoo = [new Dog("Rex"), new Cat("Whiskers")] in
17 for (a in zoo) print(a.greet());

```

LISTADO 9.1. Herencia, polimorfismo y vectores heterogeneos.

9.2.2 Protocolos con conformidad estructural

```

1  protocol Printable {
2      to_str(): String;
3  }
4
5  type Point(x: Number, y: Number) {
6      x = x; y = y;
7      to_str(): String => "(" @ x @ ", " @ y @ ")";
8  }
9
10 function show(item: Printable) => print(item.to_str());
11 show(new Point(3, 4));

```

LISTADO 9.2. Protocolos y conformidad implícita.

9.2.3 Vectores y generadores

```

1  function fib(n: Number): [Number] =>
2      let result = [0 | i in range(0, n)] in {
3          result[0] := 0; result[1] := 1;
4          for (i in range(2, n)) result[i] := result[i-1] + result[i-2];
5          result
6      };
7
8  for (x in fib(10)) print(x);

```

LISTADO 9.3. Cálculo de los primeros 10 Fibonacci con generadores.

10

Decisiones de diseño significativas

TODO compilador real es una secuencia de decisiones de compromiso entre simplicidad de implementación, eficiencia del código generado y completitud del lenguaje soportado. Este capítulo documenta las decisiones más significativas tomadas durante la construcción del compilador y articula explícitamente las alternativas descartadas, los argumentos a favor de la opción elegida y los costos asumidos.

10.1 LALR(1) artesanal frente a generador externo

El parser está implementado desde cero —gramática como estructura de datos, tablas LALR(1) construidas programáticamente en tiempo de inicialización— en lugar de generarse a partir de YACC, Bison, LALRPOP o ANTLR.

Argumento principal

Transparencia algorítmica. La construcción del autómata LR(0), el cómputo de FIRST y FOLLOW, la propagación de *lookaheads* y la construcción de las tablas ACTION/GOTO constituyen el núcleo técnico de la teoría de *parsing* [1, 3]. Una implementación que delegue ese núcleo en una herramienta externa convierte al *parser* en una caja negra, lo que va en contra del principio de diseño D1 enunciado en la introducción.

Argumento secundario

Inspeccionabilidad y depurabilidad. La gramática es una estructura de datos en memoria que puede imprimirse, recorrerse y analizarse en tiempo de ejecución. Esto resulta determinante a la hora de diagnosticar conflictos *shift/reduce*: el conflicto único de la gramática (asociado al separador | de las expresiones generadoras) pudo identificarse y resolverse inspeccionando la tabla generada, sin necesidad de recompilar el compilador entre iteraciones.

Coste

Implementación mas larga (aproximadamente 800 LOC adicionales solo para el módulo LALR) y mayor tiempo de *startup* debido a que las tablas se construyen en cada invocación. Este *overhead* es de pocos milisegundos en programas típicos.

10.2 LLVM como backend vs interprete vs VM propia

La especificación permite generar código máquina directamente, usar LLVM, o implementar una máquina virtual propia.

Argumento principal

Calidad del código generado. LLVM produce código comparable al de gcc -O2 para es-

estructuras comunes. Una máquina virtual propia o un interprete serian varios ordenes de magnitud mas lentos para programas con aritmética intensiva.

Argumento secundario

Portabilidad. El mismo IR de LLVM puede compilarse para x86-64, ARM, RISC-V o WebAssembly cambiando un solo parámetro en TargetMachine.

Coste

Curva de aprendizaje de LLVM y dependencia de libLLVM-17.so en el sistema de build. El paquete *inkwell* mitiga la primera dificultad.

10.3 Vtables vs tagged union vs boxed types

Para implementar el polimorfismo dinámico se evaluaron tres alternativas:

1. **Tagged union.** Cada objeto es una union discriminada con un tag de tipo. Simple pero ineficiente: el tamaño de cada objeto es el del tipo mayor de la jerarquía.
2. **Boxed types con dispatch dinámico por nombre.** Cada llamada a método consulta una tabla *hash* por nombre del método. Funciona pero introduce *overheads* de *hashing* y posibles colisiones.
3. **Vtables (elegida).** Cada objeto tiene un puntero a una tabla de funciones. Es la técnica usada por C++, Java JIT, y Rust (para *trait objects*). El costo por llamada es una carga adicional desde cache; el beneficio es el layout estático y conocido.

10.4 Type tags DFS pre-orden vs class objects

El test de tipo `is` se podría implementar de varias formas:

1. **Recorrer la cadena de herencia** en *runtime* comparando nombres o punteros a *class objects*. $O(h)$ donde h es la altura de la jerarquía.
2. **Tabla de class objects.** Almacenar en cada objeto un puntero a su *class*, y consultar una tabla precomputada de relaciones. $O(1)$ con espacio cuadrático en el número de tipos.
3. **Type tags DFS pre-orden (elegida).** $O(1)$ en tiempo, $O(N)$ en espacio.

La elección DFS pre-orden es la combinación óptima tiempo-espacio para el caso común.

10.5 Enlace dinámico vs estático de LLVM

El compilador enlaza dinámicamente con libLLVM-17.so. La alternativa (enlace estático) produciría ejecutables de 50–200 MB. El enlace dinámico mantiene el binario en pocos megabytes y comparte la biblioteca con otras herramientas del sistema.

11

Evaluación y pruebas

LA validación del compilador se realiza mediante una batería de pruebas automatizadas que cubren todas las fases y todas las construcciones del lenguaje. Esta sección describe la estructura de las pruebas y los resultados obtenidos.

11.1 Estructura de la suite de pruebas

TABLA 11.1. Distribución de las pruebas automatizadas por modulo.

MODULO	TESTS	COBERTURA
Lexer	87	Todos los tipos de token, edge cases, escapes
Parser	112	Todos los no-terminales, precedencia, asociatividad
Semántico	98	Inferencia, protocolos, errores de tipo
Codegen	87	Aritmética, flujo, OOP, vectores, rangos, builtins
Total	384	

11.2 Programas de prueba completos

Mas alla de las pruebas unitarias, el repositorio incluye 22 programas HULK completos en `tests/hulk/`, organizados por categoría:

- `ok/minimal/`: 9 programas (aritmética, condicionales, funciones, hola mundo, `let-in`, cadenas, ciclos).
- `ok/oop/`: 3 programas (clase básica, herencia, mutación).
- `ok/types/`: 3 programas (anotaciones, builtins, inferencia).
- `ok/extras/`: 2 programas (`for`, `range`).
- `errors/`: 5 programas que deben fallar con un mensaje específico.

Cada programa se compila y ejecuta, comparando la salida estandar contra una salida esperada. Los programas de error verifican adicionalmente que el código de salida sea distinto de cero y que el mensaje siga el formato del contrato de interfaz.

11.3 Contrato de errores

El compilador respeta el contrato de interfaz asociado al lenguaje: los errores se reportan por `stderr` con el prefijo `!` y el formato `(línea,col) FASE: mensaje`; el proceso termina

con código de salida $\neq 0$.

12

Limitaciones conocidas y trabajo futuro

NINGUN compilador es completo. Este capítulo documenta las limitaciones identificadas en la implementación actual y las direcciones de trabajo futuro consideradas.

12.1 Limitaciones actuales

Funciones anónimas (lambdas). La especificación (sección A.13) describe lambdas como funciones de primera clase. La implementación actual solo soporta funciones con nombre. La adición de lambdas requeriría *closures* con captura del entorno léxico, lo que implica un cambio en la representación de los valores funcionales: cada lambda debería representarse como un par (puntero a función, puntero a entorno capturado), con la correspondiente gestión de tiempo de vida sobre el entorno. La adición es mecánica pero no trivial: el código actual del codegen y el del semántico están contruidos bajo el supuesto de que toda función es global, y ese supuesto permea muchas estructuras de datos.

Procesamiento de secuencias de escape. Como se anticipó en el análisis léxico, las secuencias `\n`, `\t`, `\"` y `\\` no se descodifican y aparecen literalmente en la salida. La implementación es breve (un bucle sobre los caracteres del lexema), pero requiere decidir cuidadosamente dónde se realiza la descodificación: hacerlo en el *lexer* desincroniza la posición del lexema con la del fuente original, lo que dañaría la calidad de los mensajes de error; hacerlo en la fase semántica preserva la posición pero implica recorrer el lexema dos veces (una para descodificar, otra para construir el literal). Esta es una de las tareas pendientes de menor complejidad y mayor visibilidad para el usuario.

Gestión de memoria. El compilador adopta una estrategia de *solo-asignación*: las cadenas y los objetos se asignan en *heap* con `malloc` y nunca se liberan. Para programas con muchas operaciones de concatenación o asignación de objetos, esto produce un crecimiento monótono del consumo de memoria. Una mejora futura podría ser un *arena allocator* de *regiones* ligado al `let-in` —libera todas las cadenas creadas dentro de un `let` al salir de él—, o un sistema de *reference counting* con detección de ciclos similar al de Python.

Recuperación de errores en el parser. El parser se detiene en el primer error sintáctico. Un parser con recuperación podría reiniciar en el siguiente punto de sincronización (`;` o `}`) y reportar todos los errores en una sola pasada. La técnica clásica de Burke y Fisher [14] es directamente aplicable a la tabla generada por el constructor LALR(1) sin necesidad de regenerar la gramática.

Spans en errores de codegen. Los errores internos del generador de código no incluyen la posición en el fuente. Las fases anteriores propagan Span en todos los nodos del AST; propagarlos hasta el codegen mejoraría la experiencia de diagnóstico en caso de errores internos del compilador.

Vectores sin mutación estructural. Los vectores son de tamaño fijo: una vez creados, no admiten push ni pop. Agregar mutación estructural requeriría sustituir el layout actual (un puntero al buffer con cabecera de tamaño) por una estructura de *capacity-and-length* similar al `Vec<T>` de RUST, con una estrategia de redimensión exponencial para amortizar el coste.

12.2 Trabajo futuro

- **Closures y lambdas:** representar funciones de primera clase con un par (*puntero a función, puntero a entorno*).
- **Generadores perezosos:** variantes lazy de las expresiones generadoras para permitir colecciones conceptualmente infinitas.
- **Inferencia Hindley-Milner completa:** el sistema actual infiere tipos a través de expresiones simples pero no resuelve constraints polimorficos complejos. H-M permitiría funciones genéricas como `function map(v: [A], f: A -> B): [B]`.
- **Optimizaciones específicas de HULK:** *inlining* de métodos monomorficos, análisis de escape para objetos de corta vida (sustituyendo *heap* por *stack*).
- **Backend WebAssembly:** cambiar el `TargetMachine` de LLVM para emitir WebAssembly permitiría ejecutar programas HULK en el navegador.

13

Conclusiones

EL compilador descrito en este reporte implementa el lenguaje HULK de extremo a extremo: a partir de un fichero fuente `.hulk`, produce un binario ELF x86-64 que el sistema operativo carga y ejecuta sin más dependencias que `libc`, `libm` y la biblioteca de *runtime* aportada por la propia distribución. Las características nucleares del lenguaje —expresiones aritméticas y de cadena, control de flujo expresivo, funciones de primer orden, tipos con herencia simple, protocolos con conformidad estructural variante y operadores `is/as` de prueba y conversión de tipos— quedan implementadas en su totalidad, junto al sistema de iterables y la extensión propia de vectores con expresiones generadoras desarrollada en el Capítulo 8.

Tres decisiones técnicas estructuran la implementación. La primera —construir el *frontend* a mano en lugar de generarlo a partir de una herramienta externa— privilegia la transparencia del autómata LALR(1) y permite resolver en el constructor de la tabla un conflicto deliberado que ninguna herramienta genérica acomoda con elegancia (el separador `|` de las expresiones generadoras, idéntico al operador OR lógico). La segunda —asignar etiquetas de tipo en orden DFS pre-orden sobre la jerarquía de herencia— reduce el operador `is` a una comparación entera de dos cláusulas, una mejora que sitúa el coste del despacho de tipos por debajo del de cualquier comprobación implementada como recorrido de tabla. La tercera —resolver el despacho a través de protocolos mediante un *switch* sobre el `type_tag`, con ramas terminadas en llamadas estáticas— contrasta con el mecanismo habitual en C++ y Rust (tablas adicionales por interfaz) y aprovecha el hecho de que el verificador semántico conoce en tiempo de compilación el conjunto completo de tipos conformes a cada protocolo invocado.

La extensión de vectores con expresiones generadoras sitúa a HULK en la familia de lenguajes que ofrecen soporte de primera clase para transformaciones declarativas de colecciones, junto a Python, Haskell, Scala y, más recientemente, C++20. Las diferencias respecto a esos cuatro lenguajes —y documentadas con detalle en la Cuadro 8.1— no son tanto de expresividad como de *integración*: la extensión convive con el sistema de tipos estático del lenguaje, con los protocolos estructurales y con la jerarquía de objetos sin introducir un subsistema paralelo. El precio asumido ha sido renunciar a filtros, a generadores múltiples y a evaluación perezosa; el beneficio, un *runtime* de pocos cientos de bytes y un IR LLVM que el *pass manager* optimiza con la misma eficacia que un bucle aritmético equivalente escrito a mano. Cada uno de esos compromisos está justificado por separado en la Apartado 8.6.

Las direcciones de trabajo enumeradas en el Capítulo 12 delimitan las extensiones naturales del compilador en su estado actual: la introducción de funciones de primera clase con cierres léxicos, el procesamiento de las secuencias de escape en literales de cadena, la recuperación

de errores sintácticos para reportar múltiples errores en una sola compilación, y la sustitución del esquema actual de asignación monotónica por un *allocator* basado en regiones ligadas al ámbito de las expresiones `let`. Ninguna de ellas es conceptualmente difícil; cada una representa una ampliación localizada del compilador construido.

Bibliografía

- [1] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman. *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley, 2006.
- [2] K. L. Thompson. “Programming Techniques: Regular Expression Search Algorithm.” *Communications of the ACM*, 11(6):419–422, 1968.
- [3] F. L. DeRemer. *Practical Translators for LR(k) Languages*. PhD thesis, MIT, 1969.
- [4] C. Lattner, V. Adve. “LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation.” *Proceedings of the International Symposium on Code Generation and Optimization*, 2004.
- [5] B. Stroustrup. *The Design and Evolution of C++*. Addison-Wesley, 1994.
- [6] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [7] S. Peyton Jones (ed.). *Haskell 98 Language and Libraries: The Revised Report*. Cambridge University Press, 2003.
- [8] M. Odersky, L. Spoon, B. Venners. *Programming in Scala*. Artima Press, 2008.
- [9] S. Klabnik, C. Nichols. *The Rust Programming Language*. No Starch Press, 2019.
- [10] inkwell: Safe Rust bindings for LLVM. <https://github.com/TheDan64/inkwell>.
- [11] R. Bringhurst. *The Elements of Typographic Style* (3rd ed.). Hartley & Marks, 2004.
- [12] A. Miede. *A Classic Thesis Style: An Homage to The Elements of Typographic Style*. LaTeX template, 2011.
- [13] A. W. Appel. *Modern Compiler Implementation in ML*. Cambridge University Press, 1998.
- [14] K. D. Cooper, L. Torczon. *Engineering a Compiler* (2nd ed.). Morgan Kaufmann, 2011.
- [15] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice Hall International, 1985.
- [16] R. Nystrom. *Crafting Interpreters*. Genever Benning, 2021. <https://craftinginterpreters.com/>
- [17] P. Wadler. “Comprehending Monads”. *Proceedings of the 1990 ACM Conference on LISP and Functional Programming*, pp. 61–78, 1990.
- [18] N. Wirth. “The Design of a Pascal Compiler”. *Software: Practice and Experience*, 1(4):309–333, 1971.